

Falco Rule Analysis

Enterprise DevSecOps Security Lab

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Document Type	Runtime Rule Review, Tuning, and Validation
Environment	Private Ubuntu VM, Kind Kubernetes, Falco, Prometheus, Grafana
Classification	Portfolio / Private Lab Evidence

Document Intent

This professional portfolio artifact describes the design, implementation, validation, and operational usage of runtime detection, alerting, dashboards, or response artifacts created inside the private Enterprise DevSecOps lab.

Coverage Area	Included in this document
Runtime detection	Falco event logic, validation evidence, and triage workflow
Security observability	Prometheus metrics, Grafana dashboarding, alert health checks
Response readiness	Analyst actions, containment commands, retest criteria, evidence handling
Portfolio value	Professional documentation showing detection engineering, SOC process, and cloud-native security maturity

Lab boundary: no external systems were targeted. All attack simulations, detections, and response workflows were performed inside an isolated private VM and Kind Kubernetes environment.

Executive Summary

This document analyzes the Falco detection rules and Prometheus alert signals used in the Enterprise DevSecOps lab. It explains how rule conditions, priorities, fields, outputs, and Prometheus metrics work together to detect suspicious Kubernetes runtime behavior.

Falco Rule Fundamentals

A Falco rule generally contains a rule name, condition, output message, priority, and optional tags. The condition describes the behavior to detect. The output controls the message and fields returned to downstream systems. The priority represents severity and supports triage. Rule fields allow Falco to include syscall, process, user, container, and Kubernetes context.

Rule Element	Purpose	Enterprise Review Question
rule	Human-readable detection name	Does it describe the risky behavior clearly?
condition	Boolean logic evaluated against events	Is it specific enough to reduce false positives?
output	Alert message and fields	Does it include pod, namespace, image, process, and user context?
priority	Severity indicator	Does it match business impact?
tags	Classification labels	Can SOC routing and reporting use them?

Rules and Signals Reviewed

Detection Name	Source	Priority	Purpose
Sensitive file opened for reading	Falco default/runtime rule	Warning	Detect access to files such as /etc/shadow by non-trusted processes
Privileged container activity	Runtime behavior and Kubernetes context	High/Critical	Detect risky workload configuration and escape-like behavior
FalcoRuntimeDetection	PrometheusRule	Warning	Raise an alert when Falco rule matches increase
FalcoKernelEventDrops	PrometheusRule	Critical	Detect loss of kernel event visibility
FalcoOutputQueueDrops	PrometheusRule	Critical	Detect alert delivery degradation

Observed Runtime Alert

The lab generated a controlled event by reading a sensitive file from a privileged pod context. The resulting Falco alert contained high-value context for triage: file path, process, user, container ID, container image, pod name, and namespace.

```
16:01:27.530306375: Warning Sensitive file opened for reading by non-trusted program |
  file=/etc/shadow gparent=<NA> ggparent=<NA> gggparent=<NA> evt_type=open user=root
  user_uid=0 user_loginuid=-1 process=cat proc_exepath=/bin/busybox parent=sh command=cat
  /etc/shadow terminal=0 container_id=f9383142676b container_name=attacker
  container_image_repository=docker.io/library/alpine container_image_tag=3.20
  k8s_pod_name=attacker-privileged k8s_ns_name=runtime-lab
```

```
jazz@ubuntu-endpoint: ~/dev x jazz@ubuntu-endpoint: ~/de x + v - □ ×

Falco detected privileged pod activity:
Defaulted container "falco" out of: falco, falcoctl-artifact-follow, falcoctl-artifact-install (init)
16:01:27.530306375: Warning Sensitive file opened for reading by non-trusted program | file=/etc/shadow gparent=<NA> ggparent=<NA> gggparent=<NA> evt_type=open user=root user_uid=0 user_loginuid=-1 process=cat proc_exepath=/bin/busybox parent=sh command=cat /etc/shadow terminal=0 container_id=f9383142676b container_name=attacker container_image_repository=docker.io/library/alpine container_image_tag=3.20 k8s_pod_name=attacker-privileged k8s_ns_name=run-time-lab
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence: Falco alert captured runtime sensitive file activity.

Rule-by-Rule Assessment

Rule / Alert	Strength	Tuning Need	Recommended Enhancement
Sensitive file read	High signal for secret discovery and escape attempts	Allowlist trusted security tools and init jobs	Add namespace owner and workload label enrichment
Privileged pod behavior	Strong indicator of container escape risk	Separate test namespaces from production namespaces	Block with Pod Security and alert on bypass attempts
FalcoRuntimeDetection	Broad coverage of all rule matches	Can be noisy without filtering priority	Route only Warning/Critical to paging; send Notice to dashboard
FalcoKernelEventDrops	High operational value	Requires baseline during load testing	Alert if drops persist beyond 1 minute
FalcoOutputQueueDrops	Protects detection delivery reliability	Needs downstream queue monitoring	Tie to incident if alerts are not delivered

Custom Rule Recommendations

The following example rules are appropriate for a private lab or as a starting point for a production tuning backlog. They should be tested and tuned before production deployment.

```
# Example custom Falco rule: privileged shell in Kubernetes
- rule: Privileged Shell in Kubernetes Pod
  desc: Detect shell execution inside privileged Kubernetes workload
  condition: spawned_process and container and proc.name in (sh, bash, ash) and k8s.ns.name != "kube-system"
  output: Privileged shell detected (user=%user.name command=%proc.cmdline pod=%k8s.pod.name ns=%k8s.ns.name image=%container.image.repository)
  priority: WARNING
  tags: [container, kubernetes, privilege-escalation, devsecops]

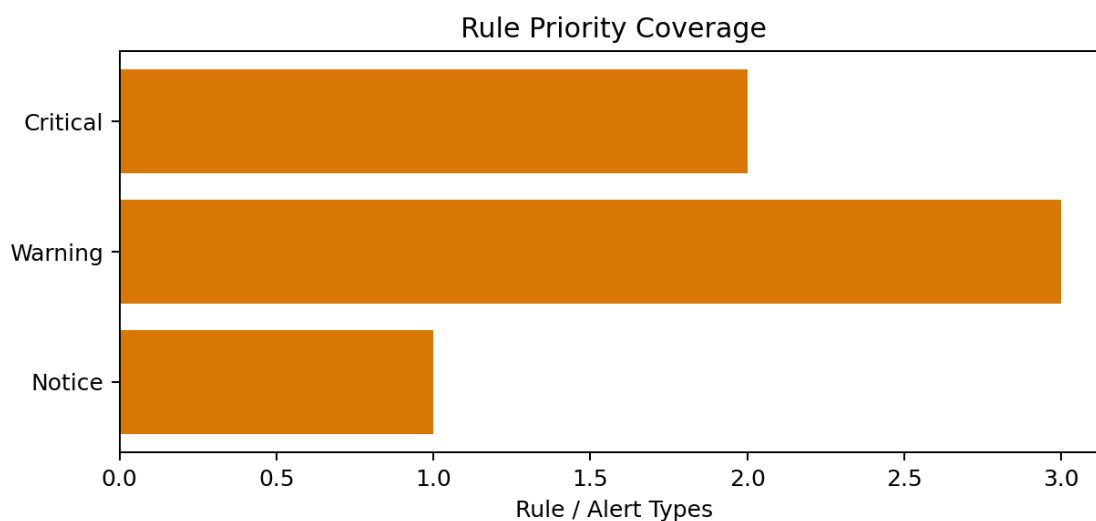
# Example custom Falco rule: host shadow read
- rule: Host Shadow File Read From Container
  desc: Detect container reading a host-mounted shadow file
  condition: open_read and container and fd.name contains "/host/etc/shadow"
  output: Host shadow read from container (file=%fd.name command=%proc.cmdline pod=%k8s.pod.name ns=%k8s.ns.name)
  priority: CRITICAL
  tags: [container, credentials, hostpath, devsecops]
```

False Positive and Exception Handling

Potential False Positive	Why It Happens	Handling Method
Security scanner reads protected files	Scanner validates host or container posture	Allowlist by image digest and namespace only
Debug shell used by platform admin	Emergency troubleshooting	Require ticket ID and time-bound exception
Backup or compliance agent opens sensitive files	Legitimate file integrity or backup process	Allowlist service account and image digest
Kernel drops during load test	Synthetic load overwhelms event processing	Tune resources and document test window

Validation Matrix

Test Case	Expected Alert	Status
Read /etc/shadow inside alpine pod	Sensitive file alert	Validated
Run privileged hostPath pod	Privileged behavior and sensitive file context	Validated
Query Falco metrics in Prometheus	Rule match counter > 0	Validated
Create dashboard panel from rule matches	Grafana panel populated	Validated



Rule priority coverage summary.

References

Falco rules basic elements - <https://falco.org/docs/concepts/rules/basic-elements/>

Falco supported fields for rule conditions and outputs - <https://falco.org/docs/reference/rules/supported-fields/>

Falco metrics - <https://falco.org/docs/concepts/metrics/>

Falco alert forwarding and JSON outputs - <https://falco.org/docs/concepts/outputs/forwarding/>

Prometheus alerting rules - https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Grafana dashboard JSON model -

<https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/>

Grafana dashboard import workflow - <https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/import-dashboards/>